



Universidade de Brasília (UNB)
Departamento de Ciência da Computação (CIC)
CIC 0105 - ENGENHARIA DE SOFTWARE - Turma 01 - 2026/1
Professor: Fernando Antonio de Araujo Chacon de Albuquerque

Alunos - Matrículas:
Caetano Korilo Fleury de Amorim - 212006737
Luis Henrique Wiltgen de Toledo - 221003968
Pedro Rafael Faria Ferreira - 190094591
Vinicius Chaves - 211060764

Architecture Notebook

Kanban Web

1. Propósito do Documento

Este documento descreve as decisões arquitetônicas significativas tomadas no desenvolvimento do Kanban Web, justificando cada escolha de design e registrando as alternativas consideradas. Serve como referência para novos desenvolvedores e para avaliação acadêmica das decisões de engenharia.

2. Visão Arquitetural

2.1 Estilo Arquitetural: Monolito em Camadas + REST API

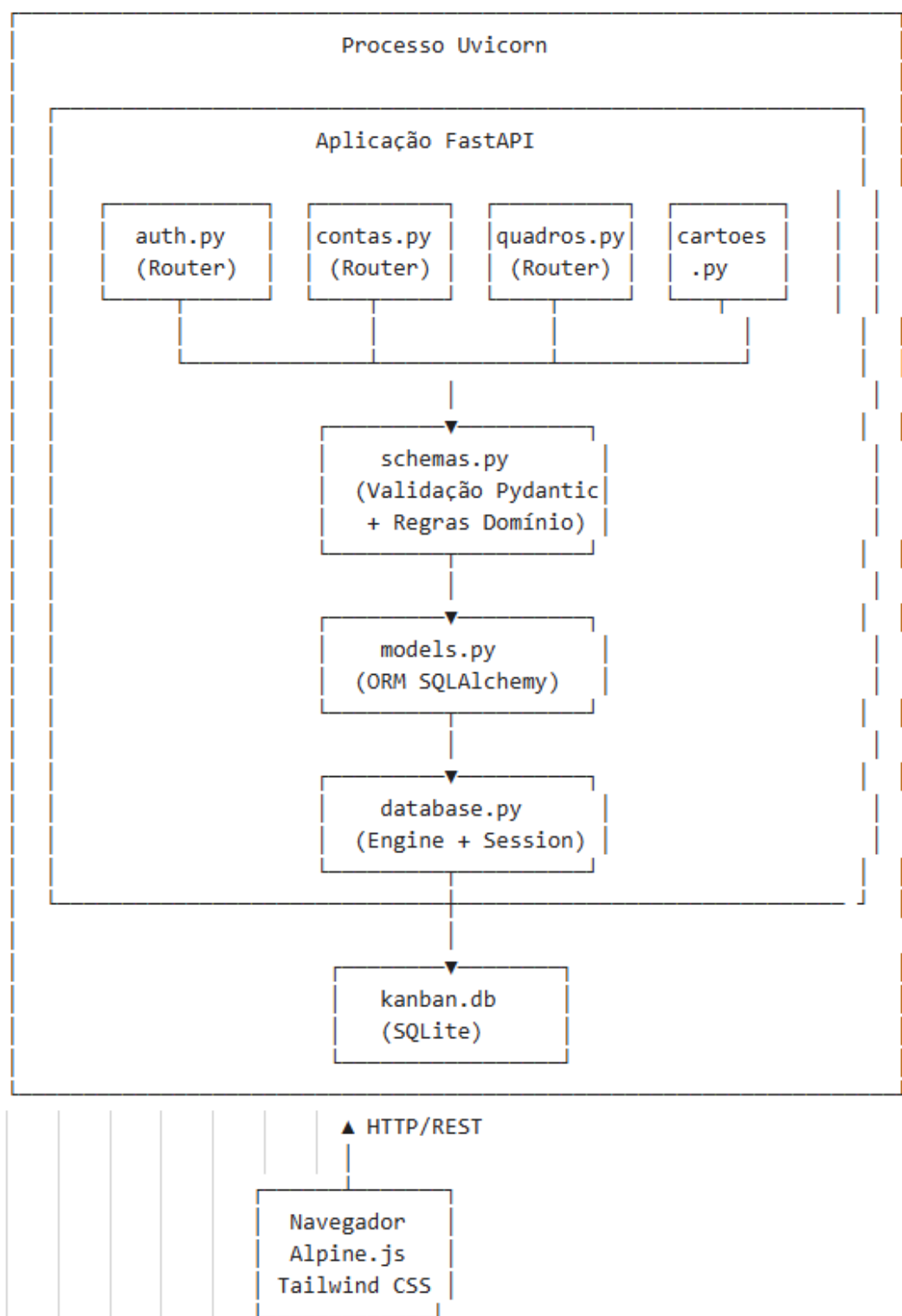
O sistema adota uma arquitetura monolítica em camadas com exposição via API REST. Esta escolha é adequada para o escopo acadêmico do projeto pelas seguintes razões:

- Simplicidade de implantação (um único processo)
- Sem overhead de comunicação entre serviços
- Fácil de depurar e testar
- Alinhado com o escopo de aprendizado da disciplina

Alternativa considerada: Microserviços - rejeitada por complexidade operacional desnecessária para este contexto.



2.2 Diagrama de Componentes



3. Camadas da Arquitetura

3.1 Camada de Apresentação (Frontend)

Tecnologia: Alpine.js + Tailwind CSS

Localização: `kanban_web/static/index.html`



Responsabilidades:

- Renderizar a interface de usuário (SPA - Single Page Application)
- Gerenciar estado local (sessão do usuário, quadro ativo, modal aberto, raias)
- Realizar chamadas HTTP à API via fetch()
- Exibir feedback visual (toast, modal, drag-and-drop em colunas e raias)

Decisões de Design:

Decisão	Escolha	Justificativa
Framework JS	Alpine.js	Leve (~14 KB gzipped), sem build step, declarativo via atributos HTML
CSS Framework	Tailwind CSS	Utilitário, sem CSS personalizado, consistência visual rápida
Comunicação	REST/JSON com fetch()	Nativo do browser, sem dependência de biblioteca HTTP
Estado	x-data Alpine.js	Estado local por componente; sem Vuex/Redux para este escopo
Roteamento	Estado booleano (quadroAtual)	SPA simples; sem necessidade de router de URL

Alternativa considerada: React + Vite - rejeitado por exigir build toolchain e complexidade desnecessária.

3.2 Camada de API (Backend REST)

Tecnologia: FastAPI

Localização: kanban_web/main.py + kanban_web/routers/

Responsabilidades:

- Receber e rotear requisições HTTP
- Aplicar middleware (CORS)
- Delegar validação ao Pydantic (schemas)
- Delegar lógica de negócio aos routers



- Servir o frontend estático

Organização dos Routers:

Arquivo	Prefixo	Responsabilidade
routers/auth.py	/auth	Cadastro e login de usuários
routers/contas.py	/contas	CRUD de conta de usuário
routers/quadros.py	/quadros	CRUD de quadros + movimentação de status
routers/cartoes.py	/cartoes	CRUD de cartões + movimentação + métricas
routers/raias.py	/raias	CRUD de raias + ordenação
routers/membros.py	/quadros	Gestão de membros (co-prefixo com quadros)

Padrões adotados:

- Cada router usa APIRouter com prefixo e tag definidos
- Todas as respostas de erro usam HTTPException com detail descritivo
- Parâmetros de usuário passados via Query string (ex: ?email_dono=...)

3.3 Camada de Validação (Schemas Pydantic)

Tecnologia: Pydantic v2

Localização: kanban_web/schemas.py

Responsabilidades:

- Validar e deserializar corpos de requisição (entrada)
- Serializar modelos ORM para JSON (saída)
- Centralizar as regras de domínio de negócio

Padrão de Schemas:

ContaCreate → validação de entrada (POST)

ContaUpdate → validação de entrada parcial (PUT)

ContaOut → projeção de saída (GET)

LoginRequest → validação de entrada de autenticação

Validadores de Domínio Centralizados:

- validar_codigo(v) - formato LLDD



- validar_email(v) - estrutura de e-mail com regras customizadas
- validar_senha(v) - complexidade mínima
- validar_texto(v) - tamanho e capitalização
- validar_nome(v) - nome de pessoa com restrições de caracteres
- validar_limite(v) - enum de valores WIP permitidos

3.4 Camada de Modelo ORM (Entidades)

Tecnologia: SQLAlchemy 2.0

Localização: kanban_web/models.py

Responsabilidades:

- Definir o mapeamento objeto-relacional das tabelas do banco
- Declarar relacionamentos e cascades
- Definir constraints de unicidade

Entidades:

Classe	Tabela	Relacionamentos
Conta	contas	1→N com Quadro (dono); 1→N com QuadroMembro
Quadro	quadros	N→1 com Conta; 1→N com Raia; 1→N com Cartao; 1→N com QuadroMembro
Raia	raias	N→1 com Quadro; 1→N com Cartao
QuadroMembro	quadro_membros	N→1 com Quadro; N→1 com Conta
Cartao	cartoes	N→1 com Quadro; N→1 com Raia

3.5 Camada de Persistência

Tecnologia: SQLite + SQLAlchemy Session

Localização: kanban_web/database.py

Responsabilidades:

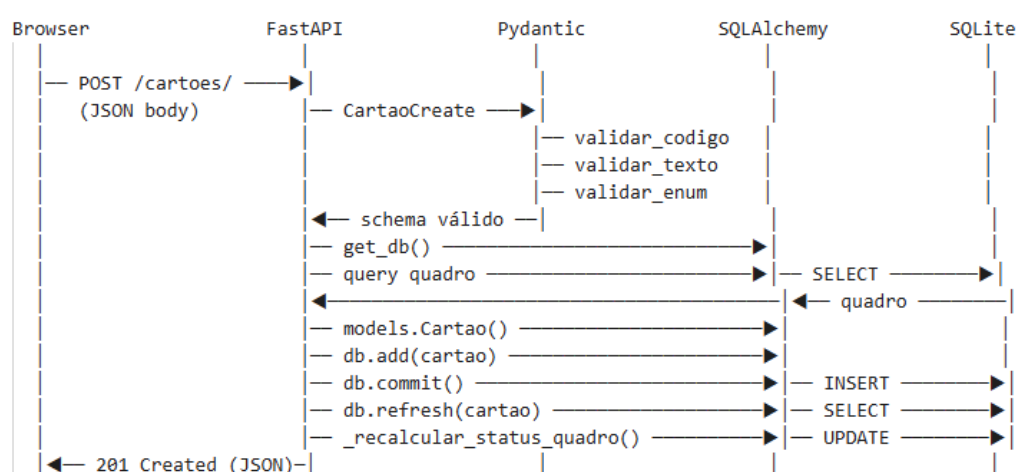
- Criar e configurar o engine do banco
- Gerenciar sessões por requisição (padrão "session per request")
- Expor a função get_db() como dependency injection do FastAPI



Padrão de Session:

```
def get_db():
    db = SessionLocal()
    try:
        yield db    # Disponibilizado ao router via Depends(get_db)
    finally:
        db.close()  # Sempre fechado ao fim da requisição
```

4. Fluxo de uma Requisição (Exemplo: Criar Cartão)



5. Decisões Arquiteturais Registradas (ADRs)

ADR-01: SQLite como Banco de Dados

Campo	Conteúdo
Contexto	O projeto é acadêmico e executado localmente
Decisão	Usar SQLite em vez de PostgreSQL ou MySQL
Razão	Zero configuração; arquivo único (kanban.db); portabilidade total
Trade-offs	Sem suporte a múltiplos escritores concorrentes; não adequado para produção em escala
Revisão	Para produção, substituir por PostgreSQL com mínima mudança (apenas a URL do SQLAlchemy)



ADR-02: Autenticação por E-mail sem JWT

Campo	Conteúdo
Contexto	Sistema acadêmico sem requisito de segurança de produção
Decisão	Autenticação por e-mail + senha, sessão mantida em localStorage
Razão	Simplicidade de implementação; sem dependência de biblioteca de JWT
Trade-offs	Sem expiração de sessão; sem token revogável; não adequado para produção
Revisão	Em produção, implementar python-jose com JWT e refresh tokens

ADR-03: Frontend como SPA Servida pelo Próprio Backend

Campo	Conteúdo
Contexto	Simplicidade de implantação com um único servidor
Decisão	FastAPI serve static/index.html na rota raiz /
Razão	Zero configuração de servidor web separado; CORS simplificado
Trade-offs	Acoplamento entre frontend e backend; não escala independentemente
Revisão	Para escala, separar em dois servidores com CORS restrito

ADR-04: Alpine.js em vez de React/Vue

Campo	Conteúdo
Contexto	Frontend simples sem roteamento complexo ou gerenciamento de estado global
Decisão	Usar Alpine.js (CDN) em vez de frameworks SPA modernos
Razão	Sem build step; arquivo único; curva de aprendizado mínima; suficiente para o escopo
Trade-offs	Dificulta componentização em aplicações maiores; menos ecossistema



ADR-05: Cálculo Automático de Status do Quadro

Campo	Conteúdo
Contexto	O status do quadro deve refletir o estado real das tarefas
Decisão	Status calculado a cada mutação de cartão via <code>_recalcular_status_quadro()</code>
Razão	Elimina inconsistência entre estado dos cartões e estado do quadro
Trade-offs	Pequeno overhead por operação de cartão; pode ser otimizado com cache em escala

ADR-06: Timestamps de Ciclo (`iniciado_em`, `concluido_em`) Imutáveis

Campo	Conteúdo
Contexto	Métricas de lead time e cycle time dependem de datas precisas
Decisão	<code>iniciado_em</code> e <code>concluido_em</code> são registrados apenas na primeira transição
Razão	Evita distorção de métricas por movimentos de ida e volta entre colunas
Trade-offs	Se um cartão voltar de FEITO para EM TESTE, o timestamp permanece

ADR-07: Raias Horizontais por Responsável

Campo	Conteúdo
Contexto	O quadro precisava explicitar distribuição de carga entre membros
Decisão	Adotar raias horizontais por responsável, com CRUD próprio
Razão	Melhora visibilidade de ownership e facilita reatribuição de trabalho
Trade-offs	Maior complexidade de UI (matriz coluna x raia) e de persistência
Revisão	Evolução futura pode permitir raias por outro critério (ex: prioridade)



ADR-08: Throughput em Janela Móvel de 7 Dias

Campo	Conteúdo
Contexto	Era necessário padronizar throughput para leitura operacional consistente
Decisão	Throughput oficial definido como cartões concluídos nos últimos 7 dias
Razão	Janela curta e estável para detectar variação recente de entrega
Trade-offs	Não representa sazonalidade de períodos longos (ex: mês/sprint completa)
Revisão	Futuramente aceitar seleção de período (7d/14d/30d) mantendo 7d padrão

Contrato de métricas por quadro:

- `lead_time_medio_dias`: média de `concluido_em` - `criado_em`
- `cycle_time_medio_dias`: média de `concluido_em` - `iniciado_em`
- `throughput_7d`: quantidade de cartões concluídos nos últimos 7 dias
- `throughput_dia_medio`: `throughput_7d` / 7
- `wip_atual`: quantidade de cartões na coluna FAZENDO

6. Mapeamento com Arquitetura C++ Original

O Kanban Web é uma evolução web do sistema C++ original. A tabela abaixo mostra a correspondência entre as camadas:

Camada C++	Camada Web
<code>CntrlApresInic / CntrlApresAuten</code>	<code>static/index.html</code> (login/cadastro)
<code>CntrlApresGeren</code>	<code>static/index.html</code> (painel Kanban)
<code>CntrlServAuten</code>	<code>routers/auth.py</code>
<code>CntrlServConta</code>	<code>routers/contas.py</code>
<code>CntrlServGeren</code>	<code>routers/quadros.py</code> + <code>routers/cartoes.py</code>
<code>ContainerConta</code>	Tabela contas (SQLite)
<code>ContainerQuadro</code>	Tabela quadros (SQLite)
<code>ContainerCartao</code>	Tabela cartões (SQLite)
<code>dominios.h</code> (validações)	<code>schemas.py</code> (validadores Pydantic)



Domínio Coluna (3 valores)	ColunaEnum (4 valores - adicionado EM TESTE)
----------------------------	--

7. Qualidade Arquitetural

7.1 Testabilidade

- Camadas desacopladas permitem substituir a sessão de banco por fixture de teste
- Pydantic schemas podem ser testados independentemente dos routers
- FastAPI suporta TestClient para testes de integração sem servidor real

7.2 Extensibilidade

- Novos domínios de negócio (ex: labels, sprints) adicionados como novos routers
- Novos campos no modelo adicionados via ALTER TABLE ou migração Alembic
- Frontend adicionado com novos x-show sem alterar o backend

7.3 Observabilidade

- FastAPI gera automaticamente documentação OpenAPI em /docs
- Erros são logados pelo uvicorn no stdout
- Endpoints retornam códigos HTTP semanticamente corretos para facilitar debugging